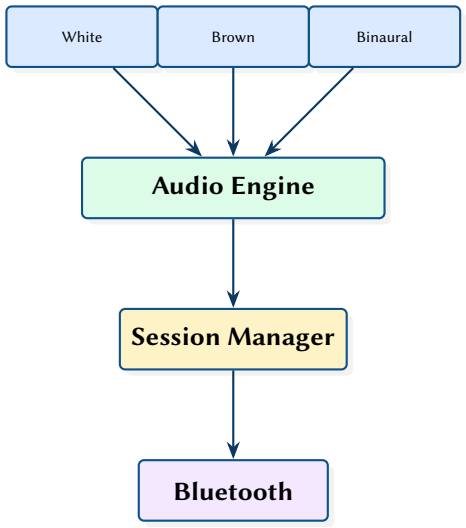
WatchNoise: Real-Time Audio Synthesis on Apple Watch

Noise Generation, Binaural Beats, and Bluetooth-Aware Playback

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 11, 2026



Abstract. WatchNoise is a minimalist Apple Watch application that generates ambient noise for sleep through Bluetooth headphones. The app synthesises white noise, brown noise (IIR low-pass filtered), smooth time-varying blends, and binaural beats at configurable frequencies using `AVAudioSourceNode` for real-time procedural generation. A four-state playback machine manages audio session lifecycle, Bluetooth routing, and automatic pause on headphone disconnect. This report covers the audio synthesis algorithms, stereo rendering architecture, and watchOS integration constraints.

Version	Date	Changes
0.1.0	March 2026	Initial architecture report

Contents

List of Design Decisions & Rules

1 Introduction and Motivation

Many people use ambient noise to fall asleep, but existing phone-based apps drain the phone battery and require keeping the phone nearby. WatchNoise runs entirely on Apple Watch, streaming audio directly to Bluetooth headphones from the wrist.

The app synthesises audio procedurally rather than playing pre-recorded loops, avoiding the perceptible repeat point that can disturb light sleepers.

: Procedural Generation Only

All audio is generated in real-time using mathematical functions. No audio files are bundled with the app. This eliminates storage overhead on the watch and provides infinite non-repeating output.

2 Audio Synthesis

2.1 White Noise

White noise is generated by sampling from a uniform random distribution and scaling to the desired amplitude:

```
func generate(into buffer: AVAudioPCMBuffer) {
    let frames = Int(buffer.frameLength)
    for i in 0..<frames {
        let sample = Float.random(in: -1.0...1.0) * volume
        buffer.floatChannelData![0][i] = sample
    }
}
```

2.2 Brown Noise

Brown noise (red noise) is produced by applying a first-order Infinite Impulse Response (IIR) low-pass filter to white noise, accumulating samples with a leaky integrator:

```
func generate(into buffer: AVAudioPCMBuffer) {
    let frames = Int(buffer.frameLength)
    let alpha: Float = 0.02  // Filter coefficient
    for i in 0..<frames {
        let white = Float.random(in: -1.0...1.0)
        lastSample = lastSample + alpha * (white - lastSample)
        buffer.floatChannelData![0][i] = lastSample * volume * 10.0
    }
}
```

The gain factor of 10.0 compensates for the low-pass filter's attenuation. The filter coefficient $\alpha = 0.02$ produces a -3 dB point at approximately:

$$f_{-3\text{dB}} = \frac{f_s}{2\pi} \cdot \alpha \approx \frac{44100}{6.28} \cdot 0.02 \approx 140 \text{ Hz} \tag{1}$$

2.3 Blended Noise

The blended generator produces a time-varying mix of white and brown noise, smoothly cross-fading over a configurable period to create a gently evolving soundscape.

2.4 Binaural Beats

Binaural beats [1] create the perception of a third tone by playing slightly different frequencies in each ear. The beat frequency equals the difference between left and right:

$$f_{\text{beat}} = |f_L - f_R| \tag{2}$$

Table 1. Binaural beat presets.

Preset	Beat Freq	Base Freq	Brain Wave
Deep Sleep	2 Hz	200 Hz	Delta
Sleep	4 Hz	200 Hz	Delta
Meditation	6 Hz	250 Hz	Theta
Relaxation	8 Hz	250 Hz	Alpha
Focus	15 Hz	300 Hz	Beta

The stereo generator renders sine waves at f_{base} (left) and $f_{\text{base}} + f_{\text{beat}}$ (right) using `AVAudioSourceNode` [2]. A composite generator optionally mixes binaural beats with background noise at configurable ratios.

> **Design Decision 1: Stereo Requirement**
>
> Binaural beats require stereo headphones to work—the effect depends on each ear receiving a different frequency. The app detects mono output routes and displays a warning, falling back to standard noise generation.

3 Playback State Machine

The `PlaybackManager` coordinates audio engine, session, and Bluetooth state through a four-state machine:

1. **Idle** — No audio session active
2. **Preparing** — Session configured, waiting for Bluetooth route confirmation
3. **Playing** — Audio engine running, source node generating samples
4. **Paused** — Engine paused (headphone disconnect or user action)

The session manager monitors Bluetooth route changes via `AVAudioSession.routeChangeNotification`. When headphones disconnect, playback automatically pauses to avoid audio leaking through the watch speaker.

4 Design Summary

Insight: *The Digital Crown is mapped to volume control via `WKInterfaceDevice.play` for haptic feedback, providing a physical volume knob without visual interaction— ideal for adjusting volume in the dark while falling asleep.*

Table 2. Key design decisions.

Decision	Rationale
Procedural synthesis	No storage overhead; infinite non-repeating audio
IIR brown noise	Single multiply-add per sample; minimal CPU on watch
AVAudioSourceNode	Real-time callback; no buffer pre-filling needed
Auto-pause on disconnect	Prevents speaker leak; respects sleep context
Digital Crown volume	Eyes-free interaction for sleep use case

References

[1] G. Oster, "Auditory beats in the brain," *Scientific American*, vol. 229, no. 4, pp. 94–102, 1973.

[2] Apple, *AVAudioEngine: Audio processing graph*, 2024. [Online]. Available: `https://developer.apple.com/documentation/avfaudio/avaudioengine`